

4803019

人工智能芯片：设计流程与实践

AI ASIC: Design and Practice (ADaP)

Fall 2026

燕博南

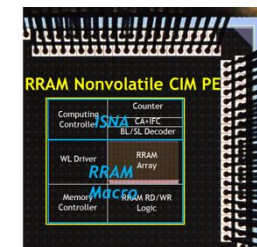
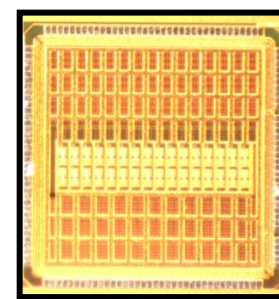
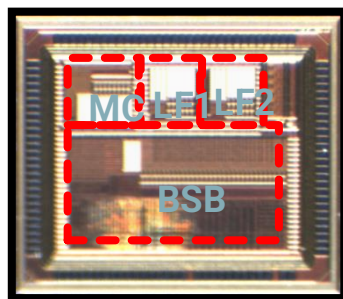
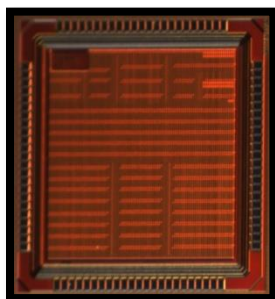
课程教授：燕博南， Ph.D.



- 2020-Now: Work as Peking University
- Education:
 - PhD Duke University, 2020
- Research:
 - In-Memory Computing Circuits & Systems
 - Domain-Specific Accelerator Chips
 - Emerging Artificial Intelligence Processor
- Bilibili: Dr燕同学



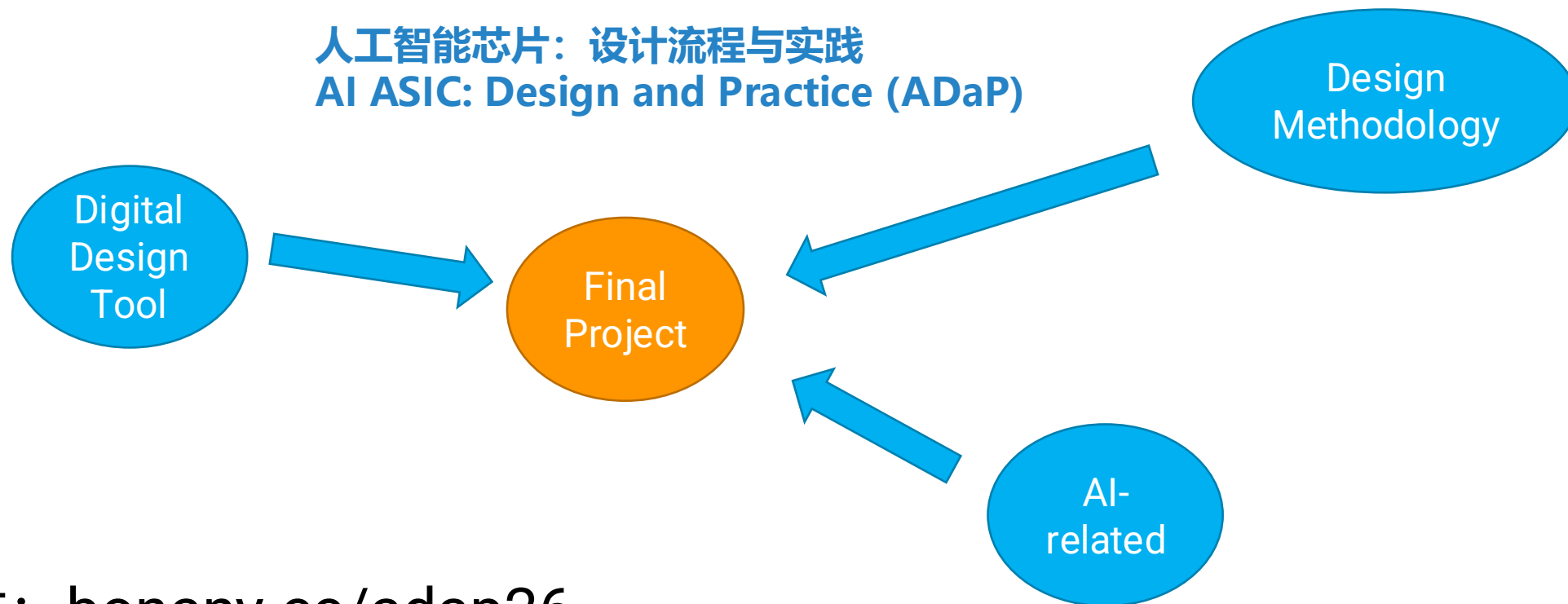
What I am working on



■ ■ 课程定位

- “研究项目” 课程

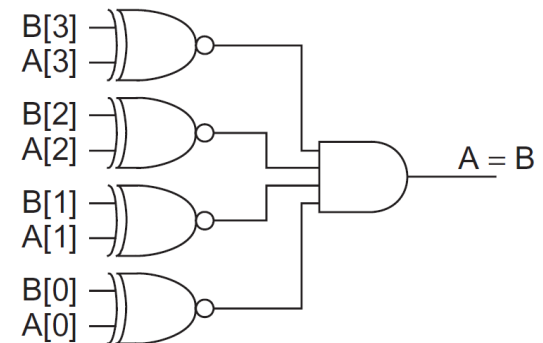
- 模式:



- 课程网站: bonany.cc/adap26

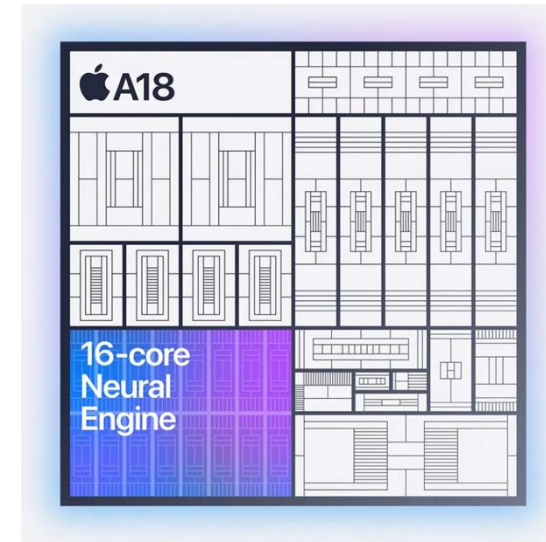
Context of ADaP

- Number 1 reason for students to enroll in ADaP:
- “Gain more experience in AI ASIC design”
- Components of AI ASIC design:
 - 1. Logic & Transistor Circuits and low-level blocks:
 - how to achieve desired function of low-level chip building blocks
 - state machines and clocking
 - performance/cost/power tradeoffs
 - physical realization concerns (floorplanning, clock distribution, pwr distribution)
 - Provides “Bottom-up” knowledge



Context of ADaP

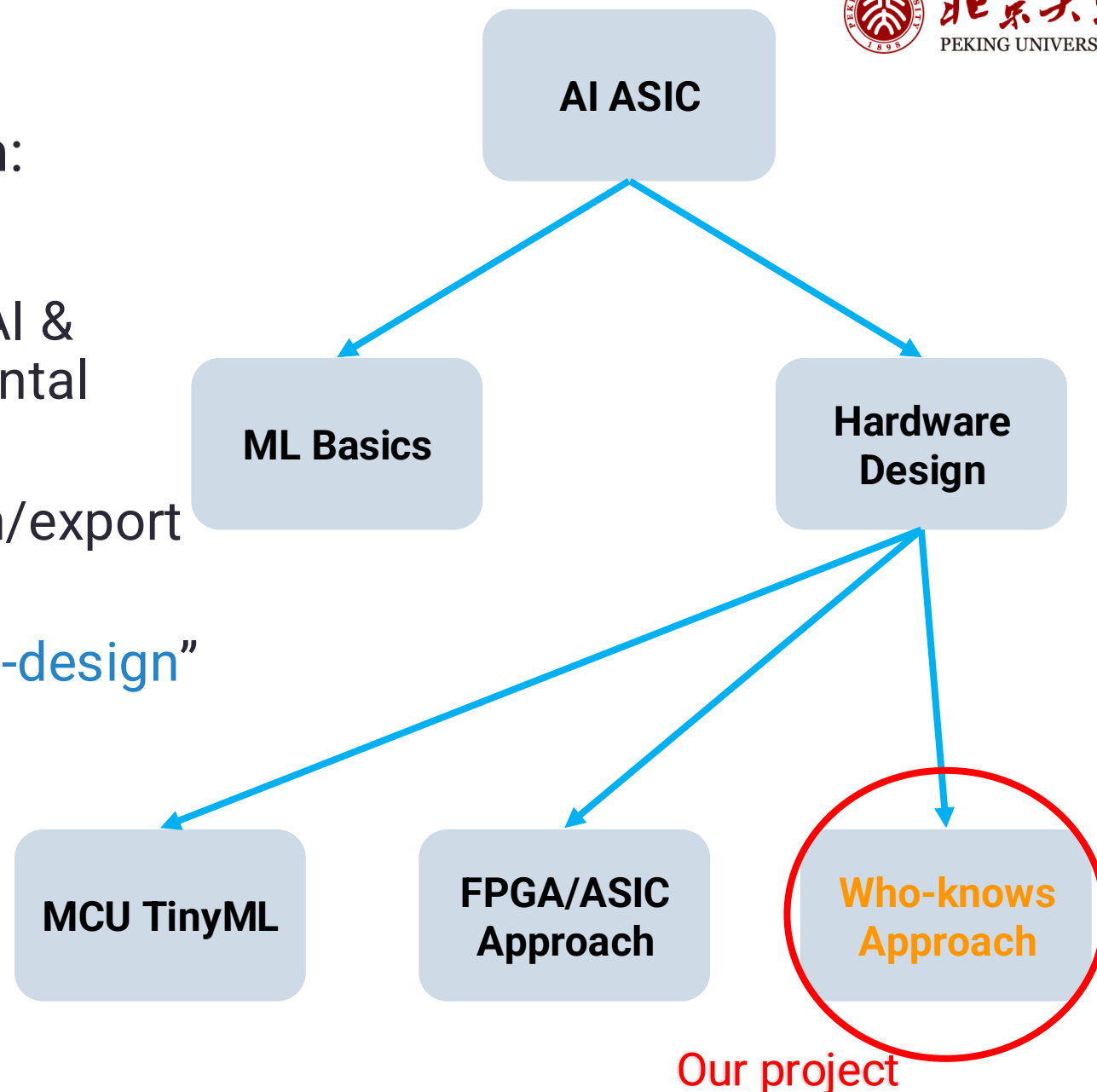
- Components of AI ASIC design:
- 2. Chip Architecture and high-level blocks:
- How building blocks are assembled to achieve high-level functionality
 - The programmable architectures start from a standard “execution model” – ISA
 - Accelerators start from an algorithm or set of algorithms.
- Provides “**Top-down**” knowledge



- **second-gen 3nm process**
- **19 billion transistors**
- **CPU: 6-core CPU, 2 high-performance cores, and 4 high-efficiency cores**
- **GPU: 5-core with support for ray tracing**
- **Neural Engine: 16-core, 35 trillion operations per second**

Context of ADaP

- Components of AI ASIC design:
- 3. Going Intelligent
- Instrumentalism perspective of AI & Machine Learning (ML) fundamental knowledge
- Use Pytorch framework to obtain/export models
- Provides “Software-Hardware Co-design” knowledge



■ ■ 后续课程安排

- 3次课讲解 -> 扼要基础知识
- 包括一次AMD-Xilinx研发科学家的教学讲座
- 9次课值日讲解 (2 ~ 3人一小组)
 - 每次一类AI芯片, 架构讲解, 最终课程全班合著一篇AI芯片架构的综述文章
 - 提炼基础知识点讲解
 - 全班一起讨论 (最后final project怎么做) +小组讨论动手实践
- 1~2次课最终汇报总结, 最终提交代码与报告 (每组一份)

AI芯片架构

基础知识点讲解

讨论分工+下手码代码

值日讲解：AI芯片架构

- TPU
- Spatial Architecture (Eyeriss) & CNN ASICs
- GPU
- TrueNorth / Loihi
- RISC-V " Vector Extension"
- Dataflow: Plasticine & Tenstorrent
- Wafer-scale
- Processing-In-Memory
- LLM Accelerator: How to?
- Emerging Technologies: Quantum Computing/ONN

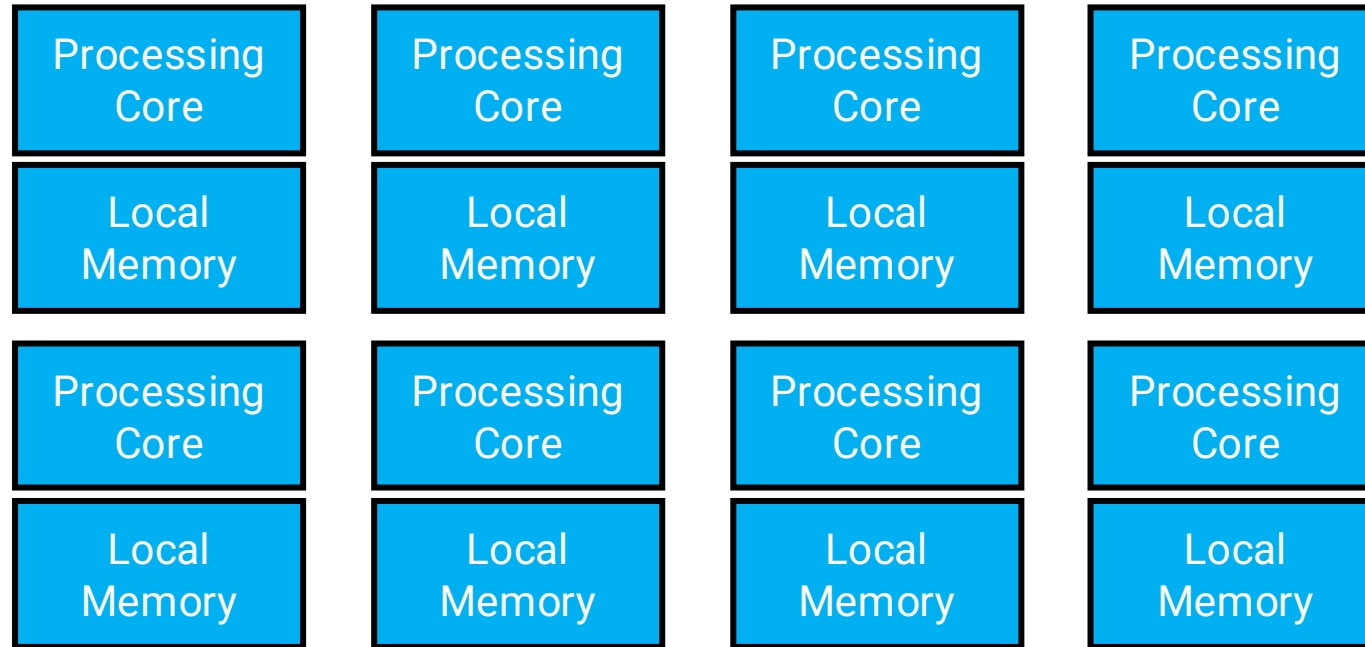
主要回答以下问题：

- 做什么计算？
- 硬件结构（架构）是什么样？
- 架构的优势是什么？
- 电路如何设计？
- 工具链是什么样的？

- 相关基础课程的关键知识点：
 - 数字电路
 - 计算机组成原理/计算机架构
 - 编译器

Final Project: Programmable AI Chip

- Group of 2~5
- Design your own AI chip & Run it!



Grading

Assignment	Assignment 1	10%	20%
	Assignment 2	10%	
Presentation	Duty Teaching	30%	40%
	Discussion on Final Project	10%	
Paper	Final Report for AI Chip Design	30%	40%
	Review paper writing	10%	

Late homework incur penalties as follows:

- Submission is 0-24 hours late: total score is multiplied by 0.9
- Submission is 24-48 hours late: total score is multiplied by 0.8
- Submission is more than 48 hours late: total score is multiplied by the Planck constant (in J·s)

评价方式：互评/相互打分

You are **encouraged** to complete the assignments/projects with peers

- ✓ discussion
- ✓ compare your answers

But DO NOT COPY

抄袭零容忍！ 后果很严重！

This course prepares you as a digital designer

- What we assume you already knows:
 - Basic digital logic
 - Know the basics of CMOS technologies
 - Python & C, knows how to code



- You will become (at least):
 - A master of Verilog HDL
 - A professional of Accelerator ASIC designer (with AI)

1. 简要自我介绍
2. 你的专业？研究方向？
3. 描述你过去的数字电路设计相关的课程
(凡是用到数字电路设计)
4. 自己有没有训练过神经网络？
5. 有没有CPU设计或应用经验？比如是否设计/仿真/
修改过各类RISC-V（蜂鸟？）、Chisel？
6. 平常用什么AI工具写代码？



- 微信群
- 发布通知（后续AI芯片架构介绍分工、最终project通知）



- End of Intro I

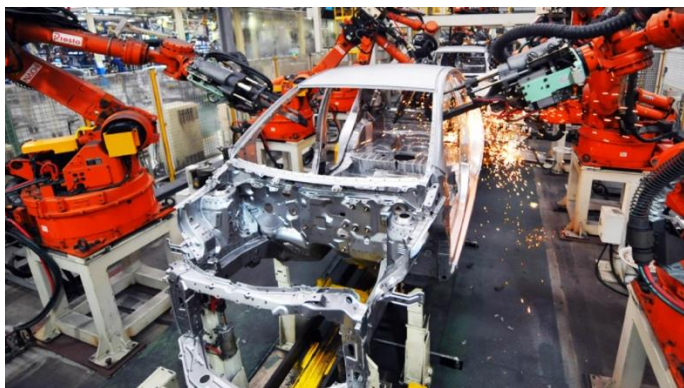
定义

Intelligence might be defined as the ability to learn and perform suitable techniques to solve problems and achieve goals, appropriate to the context in an uncertain, ever-varying world.

- A fully pre-programmed factory robot is flexible, accurate, and consistent but not intelligent.

Artificial Intelligence (AI), a term coined by emeritus Stanford Professor John McCarthy in 1955, was defined by him as “the science and engineering of making intelligent machines”.

Machine Learning (ML) is the part of AI studying how computer agents can improve their perception, knowledge, thinking, or actions based on experience or data.



In **supervised learning**, a computer learns to predict human-given labels, such as dog breed based on labeled dog pictures; **unsupervised learning** does not require labels, sometimes making its own prediction tasks such as trying to predict each successive word in a sentence;

reinforcement learning lets an agent learn action sequences that optimize its total rewards, such as winning games, without explicit examples of good techniques, enabling autonomy

Deep Learning is the use of large multi-layer (artificial) neural networks that compute with continuous (real number) representations, a little like the hierarchically organized neurons in human brains.

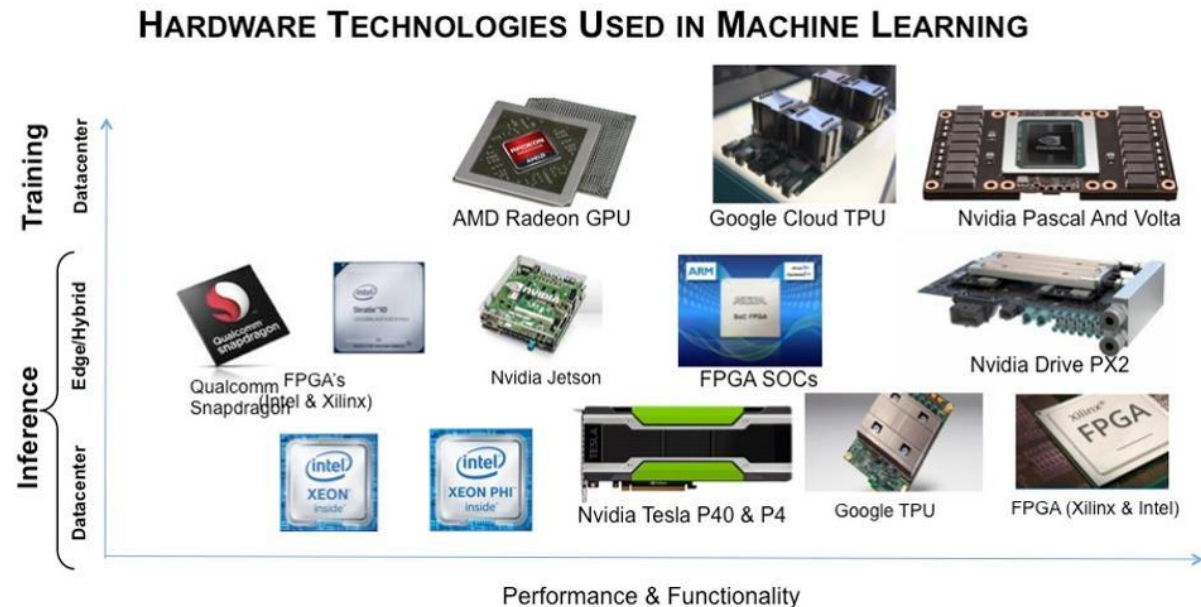
- It is currently the most successful ML approach, usable for all types of ML, with better generalization from small data and better scaling to big data and compute budgets.

Narrow AI is intelligent systems for one particular thing, e.g., speech or facial recognition.

Human-level AI, or **Artificial General Intelligence (AGI)**, seeks broadly intelligent, context-aware machines. It is needed for effective social chatbots or human-robot interaction.

为什么要有“人工智能芯片”？

- 硬件平台多种多样



Power?



Area? Volume?



Performance?



Application-Specific Integrated Circuits (ASIC)

- 硬件永远不够用!

- Energy Efficiency/Power Efficiency:
 - Unit: Op/J [operations per Joule] ~ TOPS/W
 - Unit: OPS/W [operations per second per watt] ~ TOPS/W
 - Throughput/Power
 - Peak/Average/Sparse
- Examples:
 - Processor A does INT8 Add, 1k times/second, power: 1mW, what is the energy efficiency?
 - Processor B does FP64 Multiply, 100 times/second, power: 1mW, what is the energy efficiency?

FLOPS/W

Example

Technical Specifications

	Jetson AGX Xavier Series	
	AGX Xavier	AGX Xavier Industrial
AI Performance	32 TOPS	30 TOPS
GPU	NVIDIA Volta architecture with 512 NVIDIA CUDA cores and 64 Tensor cores	
CPU	8-core NVIDIA Carmel Armv8.2 64-bit CPU 8MB L2 + 4MB L3	
DL Accelerator	2x NVDLA	
Vision Accelerator	2x 7-Way VLIW Vision Processor	
Safety Cluster Engine	-	2x Arm Cortex-R5 in lockstep
Memory	32GB 256-bit LPDDR4x 136.5GB/s	32GB 256-bit LPDDR4x (ECC support) 136.5GB/s
Storage	32GB eMMC 5.1	64GB eMMC 5.1

What is the Jetson AGX Xavier’s energy efficiency?



UPHY	8x PCIe Gen4 8x SLVS-EC 3x USB 3.1 Single Lane UFS	8x PCIe Gen4 3x USB 3.1 Single Lane UFS
Power	10W 15W 30W	20W 40W
Networking	10/100/1000 BASE-T Ethernet	
Display	Three multi-mode DP 1.2a/e DP 1.4/HDMI 2.0 a/b	
Other I/O	USB 2.0 UART, SPI, CAN, I2C, I2S, DMIC & DSPK, GPIOs	
Mechanical	100mm x 87mm 699-pin connector Integrated Thermal Transfer Plate	

- Area Efficiency:

- Unit: OPS/mm² [operations per second per mm²]
- Throughput/Area
- Peak/Average/Sparse..

Processor A does INT8 Add, 1k times/second, area: 10mm², what is the area efficiency?

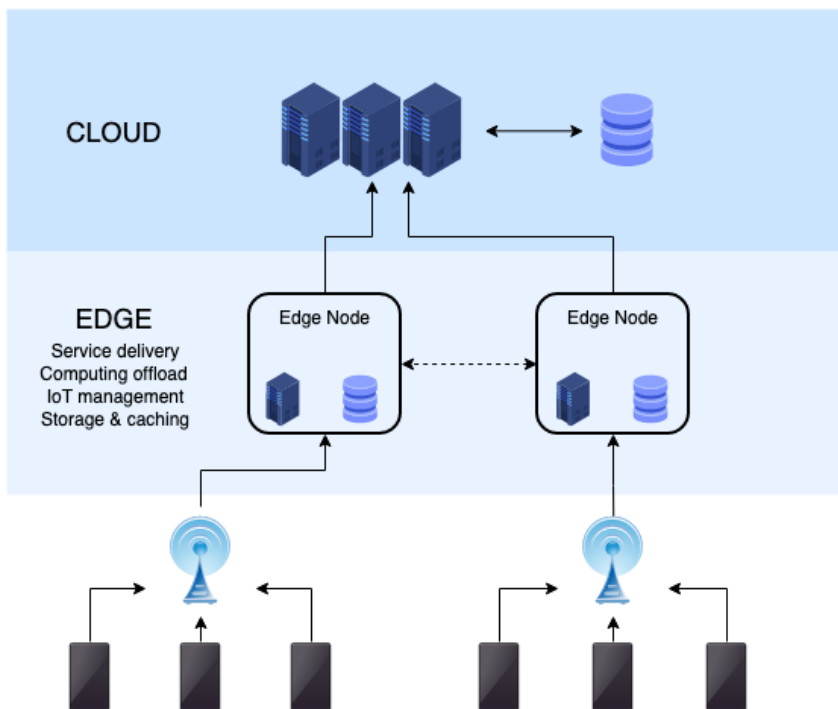
- Memory Density:

- Unit: bit/mm² [bit per mm²]
- Storage Capacity/Area

Memory A has 1Kb, area: 10mm², what is the density?

■ ■ 训练? 推理? 云? 边缘?

- 算力层次

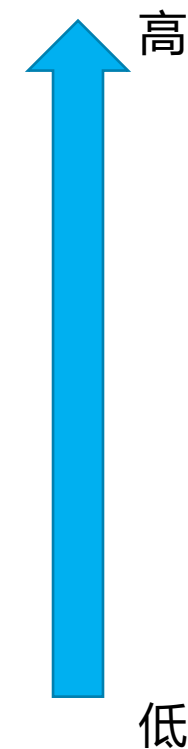


算力

规模/功耗



Training: HPC
Training
Inference: Datacenter
Inference: Edge
Inference: Mobile
Inference: Tiny (TinyML)



几个定义

- Hardcore IP: 硬核IP
 - 固定的设计, 下游开发人员不能改变的功能块
- Softcore IP: 软核
 - 用Verilog等硬件描述语言描述的功能块
- System-on-a-chip (SoC): 片上系统
 - 单个芯片上集成一个完整的系统, 一般包含
 - CPU、GPU、NPU...,
 - 总线
 - 片上存储
 - GPIO、对外的接口
- ASIC: Application-Specific Integrated Circuits 专用集成电路

SoC Example:



• 为什么用加速器?

- Domain-Specific Accelerator
- 提升算力
- 提升效率
- 相对降低成本

• 为什么用FPGA?

- 可重构
- 快速开发
- 原型设计、硬件模拟

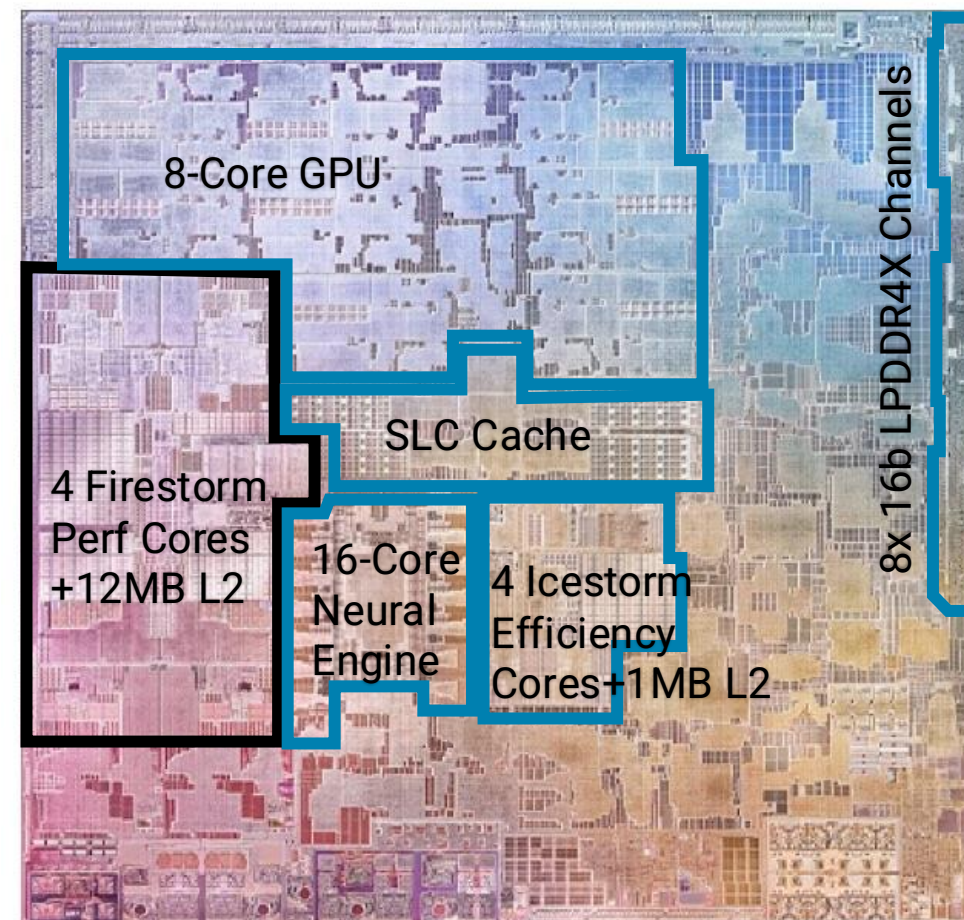
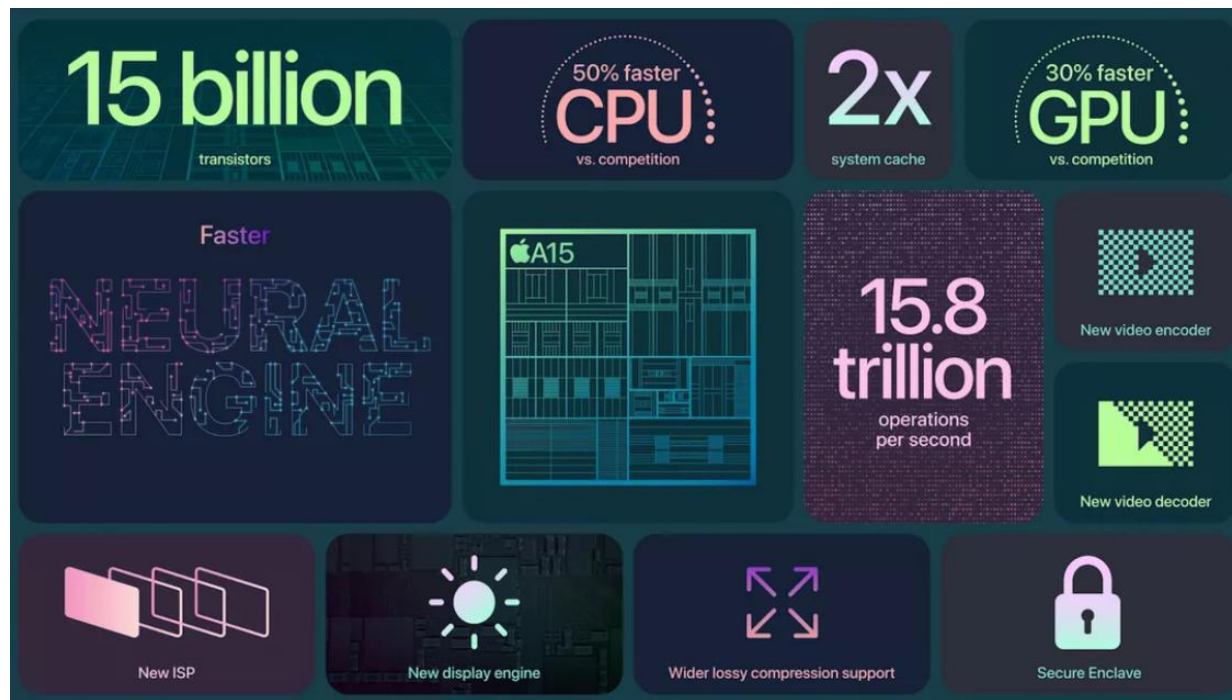
• 为什么用ASIC?

- 从最底层(Gate、Transistor)开始优化
- 灵活度高, 完全符合应用需求

FPGA/ASIC路线的哲学问题

Heterogenous Computing SoC

- Hardware accelerators
- Co-processors
- Tons of on-chip memories

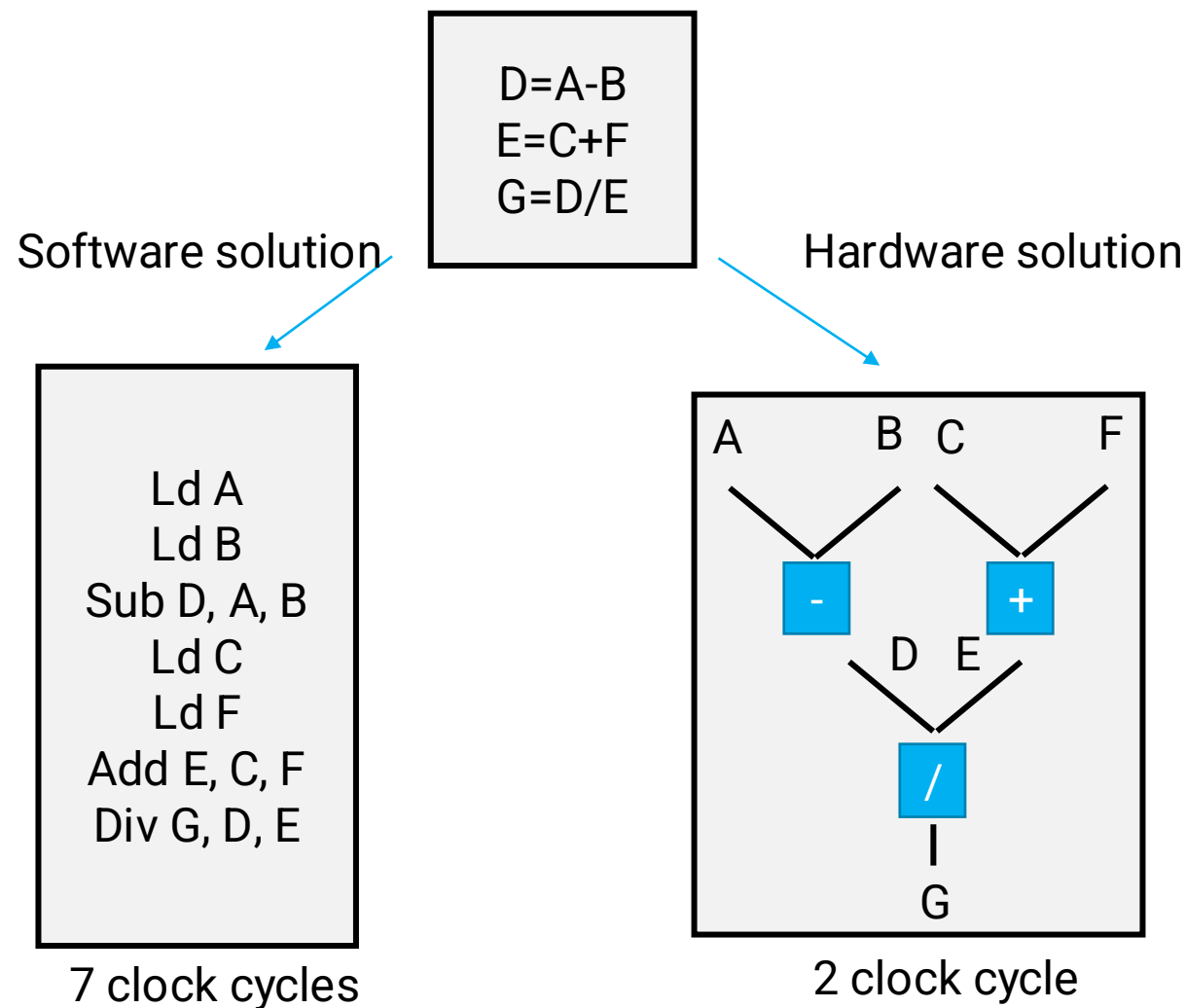


Apple M1 processor (2020)
8-core ARM, 16 billion transistors

如果你是一家SoC的架构设计师，你需要考虑...

OPs/\$ or OPs/Joule

- Exploit problem specific parallelism, at thread and instructions level
- Custom operational units or “instructions” match the set of operations needed for the algorithm (replace multiple instructions with one), custom word width arithmetic, etc.
- Remove overhead of instruction storage and fetch, ALU multiplexing

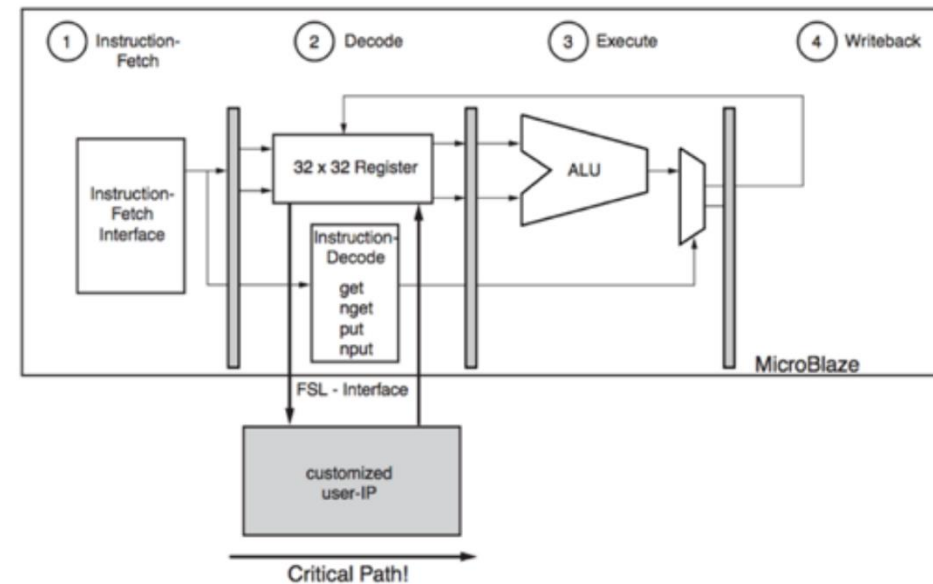
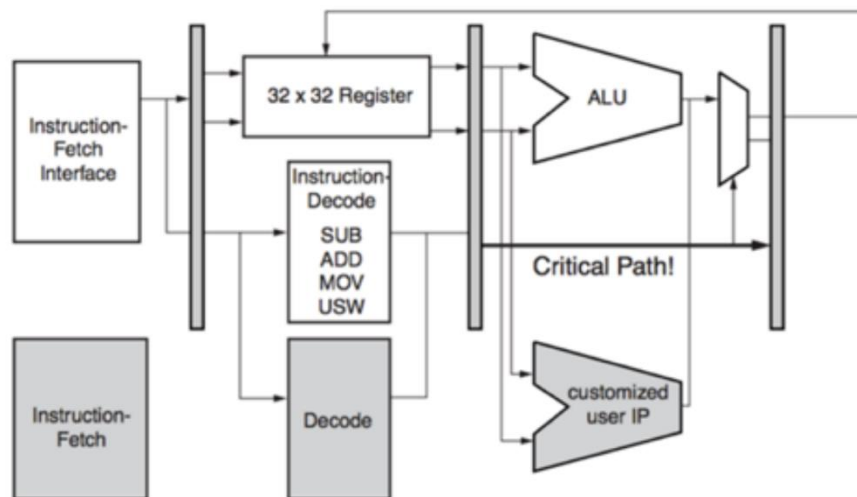




紧密连接 Tightly Coupled

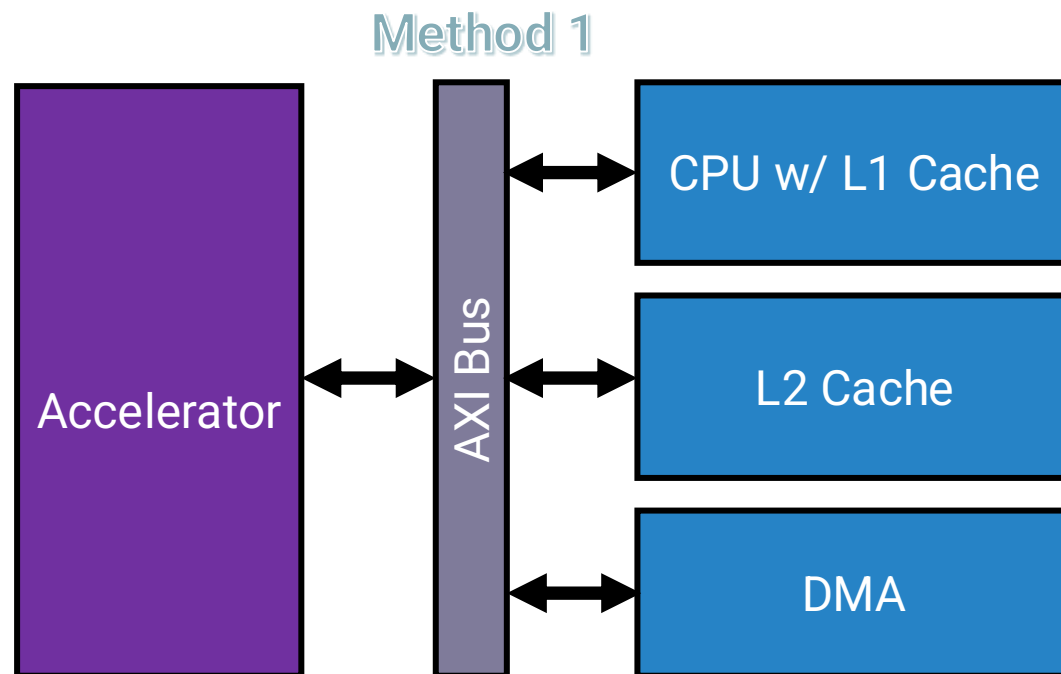
Integrated with processor control logic

- Task typically completes in a few cycles – Small amounts of data
- Processor stalls waiting for the coprocessor
- Communication with coprocessor typically via registers and dedicated control signal



Loosely-Coupled Co-processors

- Used for larger tasks than is the case for tightly-coupled coprocessors
- Task runs in parallel with main processor
- May take many cycles per task
- Large amounts of data that coprocessor may access independent of main processor – May or may not use the standard coprocessor interface

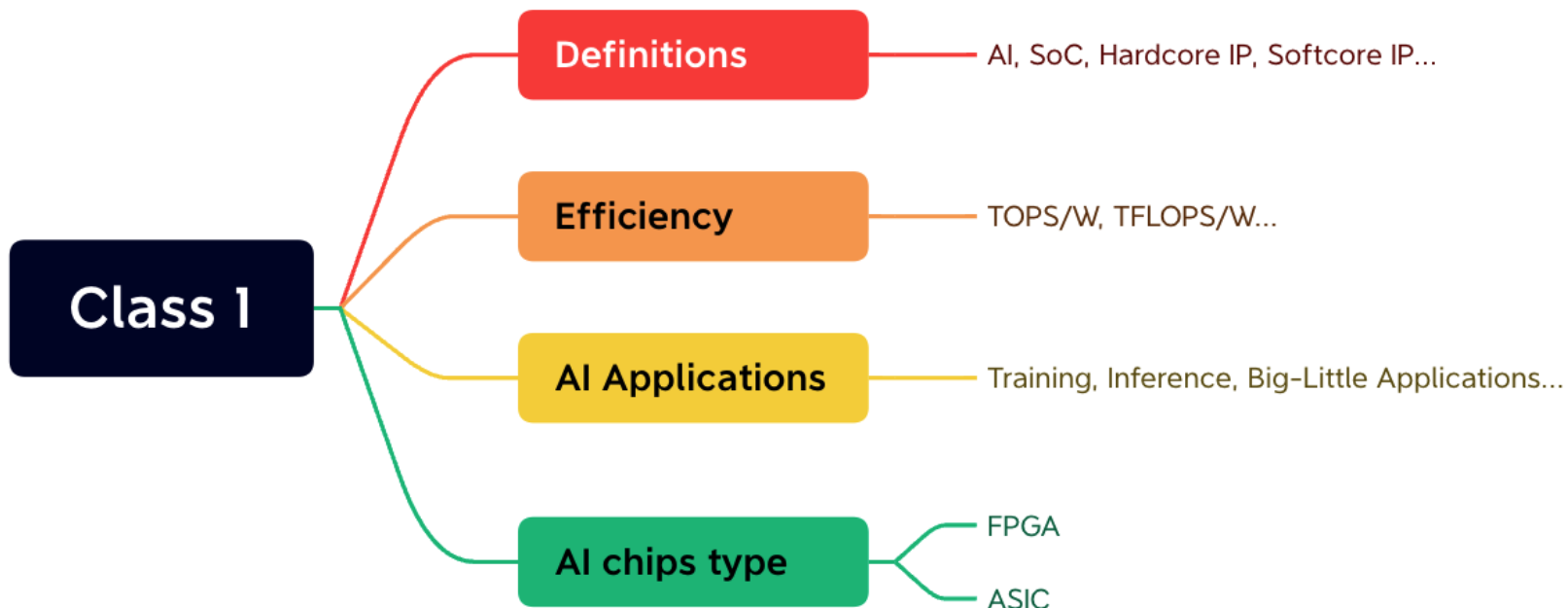




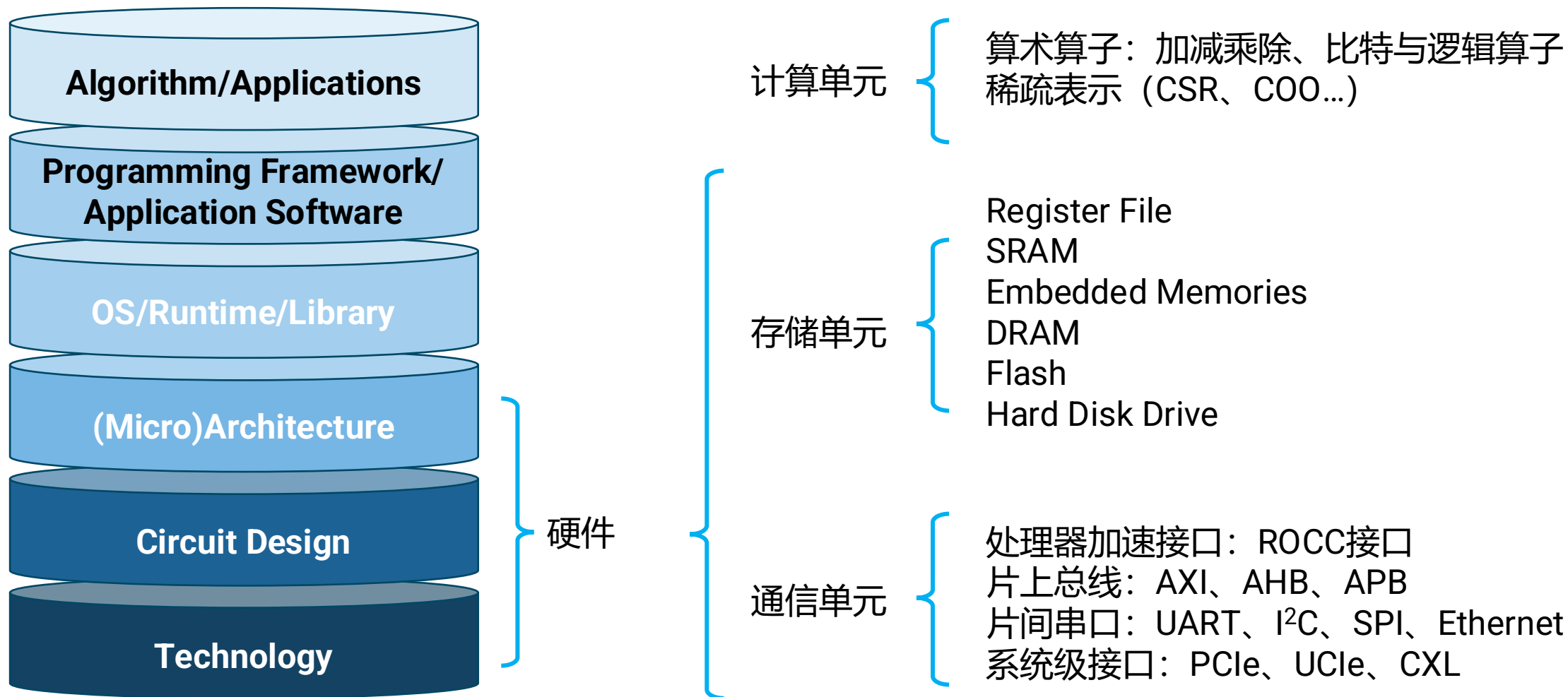
总结与思路梳理



北京大学
PEKING UNIVERSITY



知识树/分析框架



- 其它重要概念 Codesign: Design Space Exploration, System-Technology Co-Optimization (STCO), DTCO



- The End